How do we fix this? There are many ways to improve search performance so that the actual response the model retrieves is ideal or at least close to ideal. One of them is multiple negative ranking loss: Having positive pairs (query, response) & several other negative pairs (query, wrong response). Training the model to reward positive pairs & punish negative pairs.

In the current example, we would take a positive (query, response) pair, such as this one:

(Where is the world cup?, The world cup is in Qatar)

We would also take several negative (query, response) pairs such as:

- (Where is the world cup?, The world cup is in the moon)
- (" " " " " , The previous world cup was in Russia)
- (" " " " " , The world cup is in 2022)

By training the model to respond negatively to bad (query, Response) pairs, the model is more likely to give the correct answer to a query.

Now, the question is, how do we train the model to do this?, We'll see later.

(*) Multilingual Lexical Search

↳ English & Chinese have different tokenizer & stopwords.

↳ One index per language.

↳ Tools only support few languages.

improve search
he model retrieving
them is multiple
rs (query, response)
ng response).
s & punish.

tive (query-response)

in Qatar).

nse) pairs such as:

in the money)
(exp was in Russia)
is in 2022).

d (query, Response)
answer to a query
do this?.

opwords.

Multi-Lingual Embeddings.

↳ Take the text & through Transformer N/w, we get an embedding out of it & we don't need to do any language identification. & we don't need to have any stemming, stop words & so on. so everything can be set in same pipeline.

↳ Translated sentences from Neural Machine Translation:

- Issue: Models only work on sentence level.

↳ machine translated (query, answer) pairs

- Issue: Model doesn't learn language/country specific properties.

↳ We collected large quantities of training data (written by people)

- 900M English (query, answer) pairs
- 450M non-English (" ") " .

Model	Clustering (12 languages)	Search (English)	Search Non-English (16 languages)	Cross-Lingual classification (50 languages)
Cohere-Multi-Lingual	51.0	55.8	51.4	64.6
S'BERT: paraphrase-multilingual-mpnet-V2	46.7	44.4	15.3	56.1
Google: LaBSE	41.0	20.9	13.2	59.2
Google: Universal Sentence Encoder	40.1	14.3	3.4	59.8

↳ Cross-Lingual Search

Query: ما هي عاصمة الولايات المتحدة؟ (Arabic)
(What is the capital of the United States?)

Keyword Search (Elastic Search)	Semantic search with Cohere
- no results -	• Washington, D.C. [...] is the capital of the United states [...] • A capital city (or capital town or just capital) is a city or town. [...] • The United States Capitol is the building where the United states congress meets [...]

↳ Language Bias

- Preference of certain language combinations
- Language bias impacts performance negatively on multilingual pools.
- LASER & LABSE with strong language bias.

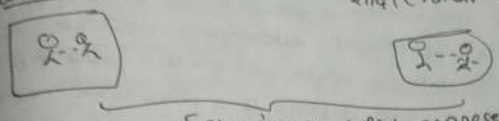
↳ Language Bias - Good or Bad?

Side-effects with language bias:

- Same language results are ranked higher just bcoz of language
- There might be better hits/answers in other languages.

Side-Effects without Language Bias.

wedding



शादी (hindi wedding)

Same image which represents wedding of 2 people according to western culture.

Q. Who is the president?
A. Joe Biden is the current President

Q. qui est le président? (French)
A. Joe Biden is the current President.

Keyword Search

We compare keyword search & dense retrieval to query a wikipedia dataset. We use Rerank to improve the results & chat to answer questions in sentence-format.

In the prev. chapter, you learned about the difference b/w keyword search to query a large dataset of wikipedia articles. Later in this same module, you'll be able to improve your results with dense retrieval & rerank, on the same wikipedia dataset, & even be able to combine the search results with a generative model in order to generate answers in the form of sentences.

Using a Vector Database

We'll setup a vector database for use in this & next chapter. In order to store the Wikipedia dataset query, we'll use the Weaviate vector database which will give us a range of benefits. In simple terms, a vector database is a place where one can store data objects & vector embeddings & be able to access them & perform operations easily. For example, finding the nearest neighbors of a vector in a dataset is a lengthy process, which is ~~speed up~~ speed up significantly by using a vector database.

This is done with the following code:

```
import weaviate
import cohere
cohere_api_key = "cohere API key"
co = cohere.Client(cohere_api_key)

# Connect to the weaviate demo ~~dataset~~ database containing
# 10m wikipedia vectors.
# This uses a public READ-ONLY weaviate API key
auth_config = weaviate.auth.AuthApiKey(api_key=
                "76320a90-53d8-42bc-b41d-
                 678647c6672e")
client = weaviate.Client(
        url = "https://cohere-demo.weaviate.network/",
        auth_client_secret = auth_config,
        additional_headers = {
            "X-Cohere-Api-Key": cohere_api_key,
        }
    )
```

Querying the

To use Keyword r
function for Keyword
vector database t
retrieved docum
language (using
in other langua

```
def Keyword_sea
    properties =

    where_fi

    }
    respons
    c

    )
    result
    return
```

Querying the Wikipedia Dataset Using Keyword Matching

To use keyword matching, we'll first define the following function for keyword search. In this function, we'll tell the vector database ~~that~~ what properties we want from each retrieved document. We'll also filter them to the English language (using results_lang), but feel free to explore searching in other languages as well!

```
def keyword_search(query, results_lang = 'en', num_results=10):
    properties = ["text", "title", "url", "views", "lang",
                  "_additional {distance}"]

    where_filter = {
        "path" : ["lang"],
        "operator" : "Equal",
        "valueString": results_lang
    }
    response = (
        client.query.get("Articles", properties)
            .with_bm25(
                query=query
            )
            .with_where(where_filter)
            .with_limit(num_results)
            .do()
    )
    result = response['data']['Get']['Articles']
    return result
```

Querying the Wikipedia Dataset Using Keyword matching

To use keyword matching, we'll first define the following function for keyword search. In this function, we'll tell the vector database ~~that~~ what properties we want from each retrieved document. We'll also filter them to the English language (using results_lang), but feel free to explore searching in other languages as well!

```
def keyword_search(query, results_lang='en', num_results=10):
    properties=["text", "title", "url", "views", "lang",
                "_additional {distance}"]

    where_filter = {
        "path": ["lang"],
        "operator": "Equal",
        "valueString": results_lang
    }
    response = (
        client.query.get("Articles", properties)
            .with_bm25(
                query=query
            )
            .with_where(where_filter)
            .with_limit(num_results)
            .do()
    )
    result = response['data']['Get']['Articles']
    return result
```

Querying the Wikipedia Dataset Using Keyword matching

To use Keyword matching, we'll first define the following function for Keyword search. In this function, we'll tell the vector database ~~that~~ what properties we want from each retrieved document. We'll also filter them to the English language (using results-lang), but feel free to explore searching in other languages as well!

```
def keyword_search(query, results_lang='en', num_results=10):
    properties=["text", "title", "url", "views", "lang",
                "_additional {distance}"]

    where_filter = {
        "path": ["lang"],
        "operator": "Equal",
        "valueString": results_lang
    }
    response = (
        client.query.get("Articles", properties)
            .with_bm25(
                query=query
            )
            .with_where(where_filter)
            .with_limit(num_results)
            .do()
    )
    result = response['data']['Get']['Articles']
    return result
```

We'll use 2 search queries, of varying difficulty.

- Simple query: "Who discovered penicillin?"
- Hard query: "Who was the first person to win two Nobel prizes?"

The responses for these queries are "Alexander Fleming" and "Marie Curie". Now let's see how keyword search does. Here are the top 3 results for each query (some results are repeated, so let's look at the 3 top distinct ones)

Query ①: "Who discovered penicillin?"

Responses:

- Penicillin
- Antibiotic
- Alexander Fleming

As you can see, keyword search did quite well. All 3 articles contain the answer, & in particular, the 3rd one is the correct response: Alexander Fleming.

Now let's see how it did with the more complicated query.

Query ②: "Who was the first person to win two Nobel prizes?"

Responses:

- Neutrino
- Western culture
- Reality television.

This time,
the answer. If
that they co
"first", "p
have any i
Nobel prize
a scientist
was not th
↳ As we can
"who discovered
to have a lo
specifically,
& "penicill
that Alex
↳ With hard
to win two
well. The
appear in
something a
prizes. By
the seman
what we
would be
what o
which is c
that are
you can
of art
reranki

This time, keyword search was very far from finding the answers. If you explore the articles, you may notice that they contain several mentions of words such as "first", "person", "Nobel", "prizes", but none of them have any info" on the first person to win two Nobel prizes. In fact, the neutrino article mentions a scientist who won two Nobel prizes, but this was not the first person to achieve this feat.

As we can see, keyword search can be good for queries, like "who discovered penicillin?", in which you'd expect the answers to have a lot of words in common with the query. More specifically, if an article contains the words "discovered" & "penicillin", it's also likely to contain the fact that Alexander Fleming discovered it.

With harder queries like "Who was the first person to win two Nobel prizes?", keyword search does not do well. The reason is that the words in the query would appear in many instances w/o necessarily talking about something as specific as the first person who won 2 Nobel prizes. By matching words, we have not yet exploited the semantics of the sentence. A model that understands what we mean by "the first person to win 2 Nobel prizes" would be able to find the answer, which is exactly what dense retrieval does. The other method is reranking, which is able to surface the pairs of queries & documents that are the most relevant to each other. In this way, you can use keyword search to retrieve, say, hundreds of articles with matching keywords & then use reranking to surface the best ones.